

Task on Load Balancers

1. Configure Classic Load balancer.

Task 1 – Classic Load Balancer

THE Configure a Classic Load Balancer to distribute incoming traffic between multiple EC2 web servers.

Steps performed:

1. Created **2 Ubuntu EC2 instances** in the same VPC.
2. Used `t3.micro` instances.
3. Enabled **Public IPv4 addresses**.
4. Configured Security Group:
 - SSH — Port `22`
 - HTTP — Port `80`
5. Installed and started **Apache2** through EC2 User Data.
6. Deployed a simple web page that displays the hostname of the responding EC2 instance.
7. Data
8. **Server 1 test:** `35.175.127.97` successfully displayed:
`Response from: ip-10-0-1-228`
9. **Server 2 test:** `44.201.45.136` successfully displayed:
`Response from: ip-10-0-1-151`
10. Opened **EC2 → Load Balancers → Create load balancer**.
11. The account currently provides **Application Load Balancer, Network Load Balancer and Gateway Load Balancer**; the Classic Load Balancer creation option is not available.

Screenshot 1 – EC2 Web Server 1

Shows Apache application successfully responding from `ip-10-0-1-228`.

Screenshot 2 – EC2 Web Server 2

Shows Apache application successfully responding from `ip-10-0-1-151`

Task 2: Create and Configure Application Load Balancer

Objective

To create an **Application Load Balancer (ALB)** and distribute incoming HTTP traffic between two Apache web servers running on EC2 instances.

Architecture

User/Browser → Application Load Balancer → Target Group → EC2 Web Server 1 / EC2 Web Server 2

Steps Performed

1. Data

- Created two Ubuntu EC2 instances:
 - **LB-Web-Server-1**
 - **LB-Web-Server-2**
- Installed and started Apache.
- Configured each server to display its hostname.
- Hand
- Both servers successfully displayed the **Load Balancer LabIn**

2. Created ALB Security Group

- Created security group: **ALB-SG**
- Allowed inbound:
 - **HTTP**
 - Port: **80**
 - Source: **0.0.0.0/0**
- This allows users from the internet to access the Application Load Balancer.

3. Created Target Group

- Opened **EC2 → Target Groups → Create target group**.
- Target type: **Instances**
- Target group name: **ALB-Target-Group**
- Protocol: **HTTP**
- Port: **80**
- VPC: **ASG-VPC**
- Protocol version: **HTTP1**

4. Registered EC2 instances

- Selected:
 - **LB-Web-Server-1**

- **LB-Web-Server-2**
 - Registered both instances on **port 80**.
 - Created the target group.
- 5. **Created Application Load Balancer**
 - Opened **EC2** → **Load Balancers** → **Create Load Balancer**.
 - Selected **Application Load Balancer**.
 - Load balancer name: **LB-Application-ALB**
 - Scheme: **Internet-facing**
 - IP address type: **IPv4**
 - Selected **ASG-VPC**.
- 6. **C**
 - S
 - **us-east-1a**→**ASG-Public-Subnet**
 - **us-east-1b**→**ASG-Public-Subnet-2**
 - This allows the ALB to operate across multiple Availability Zones.
- 7. **Configured Security Group**
 - Attached **ALB-SG** to the Application Load Balancer.
 - HTTP port **80** was allowed for incoming web traffic.
- 8. **Configured Listener and Routing**
 - Listener protocol: **HTTP**
 - Listener port: **80**
 - Default action: **Forward to target group**
 - Target group: **ALB-Target-Group**
- 9. **Created the Load Balancer**
 - Reviewed the configuration.
 - Clicked **Create load balancer**.
 - Waited until the ALB became **Active**.
- 10. **Verify**
 - Opened:
 - EC2** → **Target Groups** → **ALB-Target-Group** → **Targets**
 - Verified:
 - **LB-Web-Server-1** → **Healthy**
 - **LB-Web-Server-2** → **Healthy**
 - Result: **2 Healthy, 0 Unhealthy**
- 11. **Tested the Application Load Balancer**
 - Copied the ALB DNS name.
 - Opened the ALB DNS in the browser.
 - The application successfully displayed:

Load Balancer Lab

Response from: ip-10-0-1-228

Apache Web Server is running successfully.

Result

Application Load Balancer was successfully created and configured.

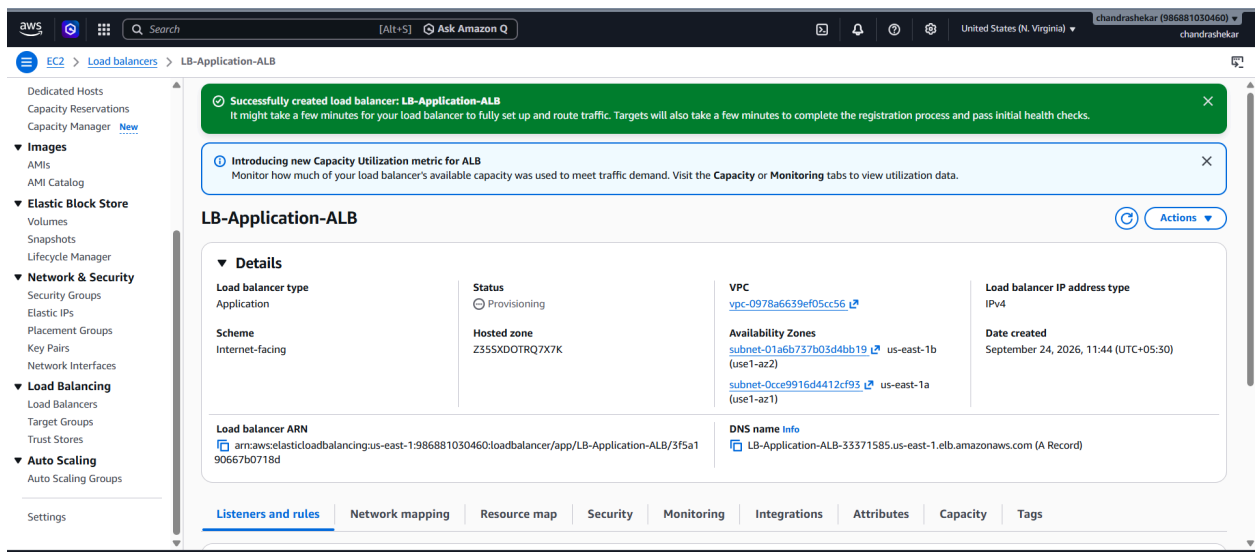
The ALB accepts HTTP requests on **port 80** and forwards them through **ALB-Target-Group** to the healthy EC2 web servers.

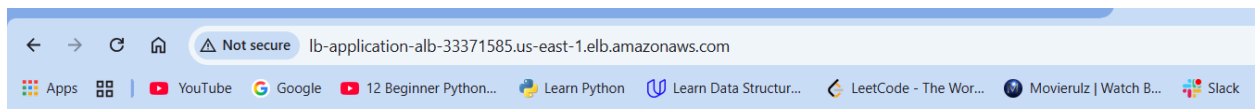
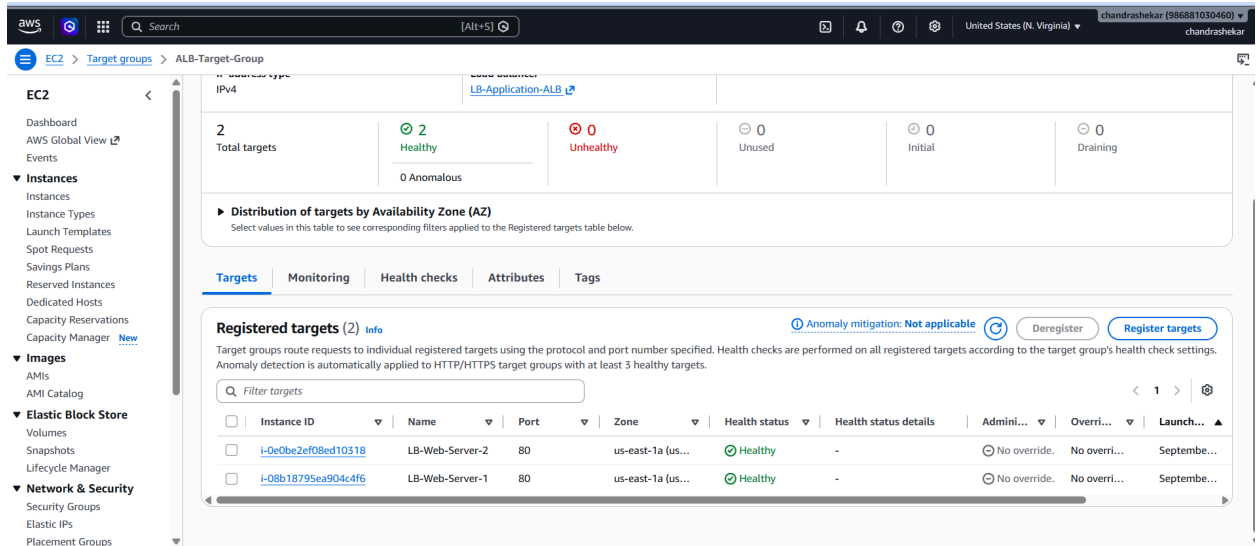
Traffic flow:

Internet User → ALB → Target Group → EC2-1 / EC2-2 → Apache Web Page

Screenshots to attach

Attach your screenshots in this order: **two individual EC2 IP tests** → **target group with 2 Healthy targets** → **ALB successfully created** → **final browser test using ALB DNS**.





Load Balancer Lab

Response from: ip-10-0-1-228

Apache Web Server is running successfully.

Final Architecture:

Internet

|

v

Application Load Balancer

HTTP : 80

|

v

ALB-Target-Group

|

+-----+

|

|

v

v

LB-Web-Server-1 LB-Web-Server-2
Apache :80 Apache :80

3. Configure Network Load balancer.

Task 3: Configure Network Load Balancer (NLB)

Objective:

Configure a Network Load Balancer to distribute TCP traffic on port 80 across two EC2 web servers.

Steps performed:

1. Opened **EC2** → **Load Balancing** → **Load Balancers** → **Create Load Balancer**.
2. Selected **Network Load Balancer**.
3. Configured basic settings:
 - **Name:** LB-Network-NLB
 - **Scheme:** Internet-facing
 - **IP address type:** IPv4
4. Selected the existing VPC:
 - **VPC:** ASG-VPC
 - **CIDR:** 10.0.0.0/16
5. Selected two public subnets for high availability:
 - us-east-1a → ASG-Public-Subnet → 10.0.1.0/24
 - us-east-1b → ASG-Public-Subnet-2 → 10.0.3.0/24
6. Selected the existing security group:
 - **Security Group:** ALB-SG
 - Port **80** allowed.
7. Created a new target group:
 - **Name:** NLB-Target-Group
 - **Target type:** Instances
 - **Protocol:** TCP
 - **Port:** 80
 - **VPC:** ASG-VPC
8. Registered the existing EC2 web servers:
 - LB-Web-Server-1
 - LB-Web-Server-2
 - **Port:** 80
9. Configured the NLB listener:

- **Protocol:** TCP
- **Port:** 80
- **Forward to:** `NLB-Target-Group`

10. Reviewed the configuration and attempted to create the Network Load Balancer.

11. AWS returned:

`OperationNotPermittedException: This AWS account currently does not support creating load balancers.`

Result:

The NLB configuration procedure was completed, but final NLB provisioning could not be completed because of an **AWS account-level restriction**.

Screenshots to attach

You only need about **3 screenshots**:

Screenshot 1 – NLB Configuration

Attach the screen showing:

- `LB-Network-NLB`
- Internet-facing
- IPv4
- ASG-VPC
- Two public subnets

Screenshot 2 – Target Group / Registered EC2 instances

Attach the screen where you selected:

- `LB-Web-Server-1`
- `LB-Web-Server-2`
- Port 80

If you have a screenshot showing the completed `NLB-Target-Group`, that's even better.

Screenshot 3 – Final configuration/error

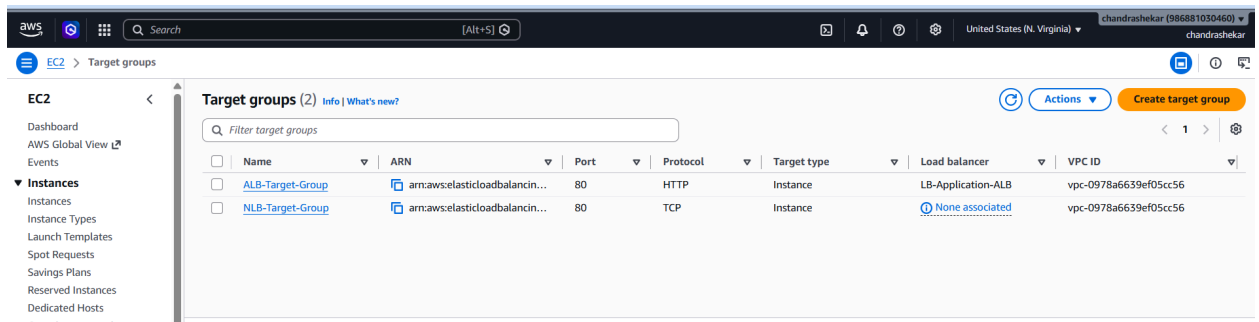
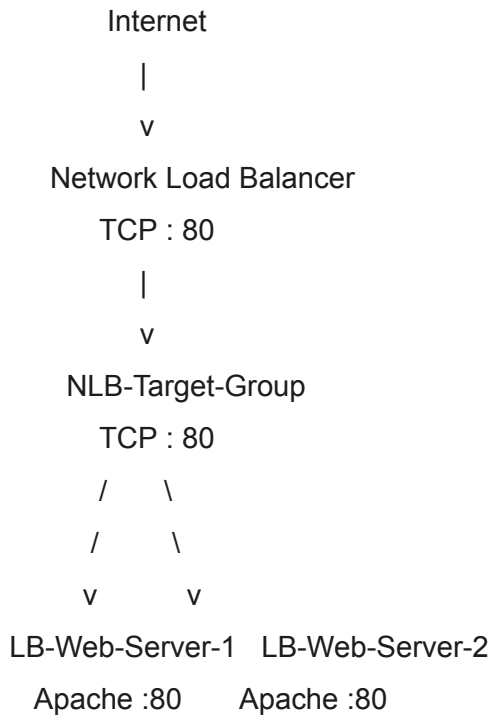
Attach the screenshot you just sent showing:

Create load balancer: LB-Network-NLB — Error
`OperationNotPermittedException...`

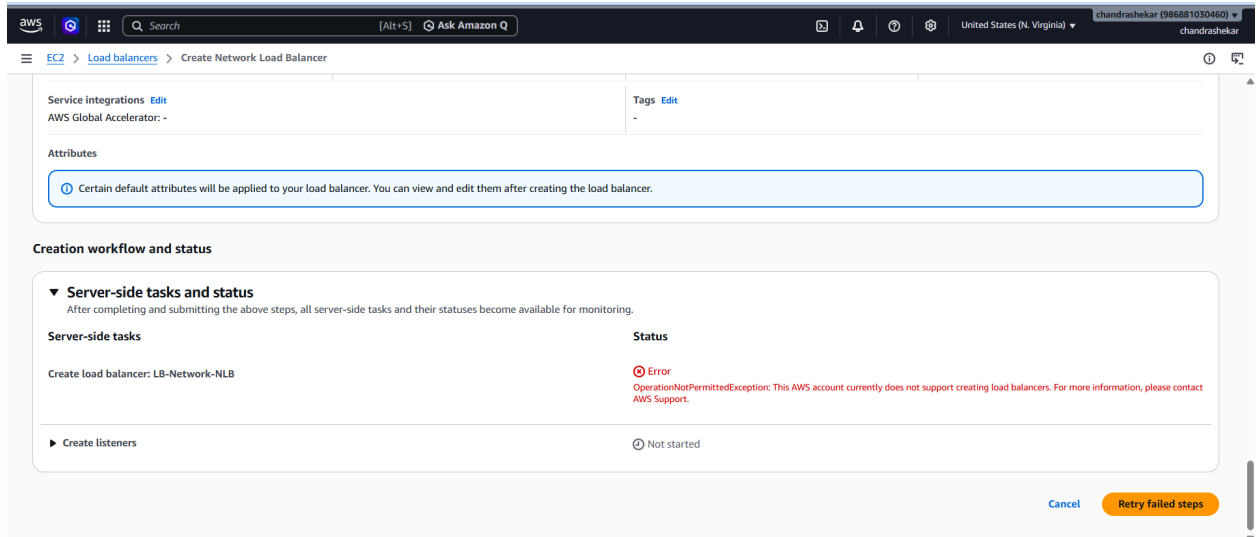
Under that screenshot write:

Note: NLB creation could not be completed due to an AWS account-level restriction. The network, listener, target group, and backend target configuration were completed as part of the lab.

Architecture to put in document



	Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
☐	ALB-Target-Group	arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/alb-target-group/12345678901234567890	80	HTTP	Instance	LB-Application-ALB	vpc-0978a6639ef05cc56
☐	NLB-Target-Group	arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/nlb-target-group/12345678901234567890	80	TCP	Instance	None associated	vpc-0978a6639ef05cc56



4. Attach SSL for application load balancer.

Task 4 – Attach SSL Certificate to Application Load Balancer, these steps:

1. Open **AWS Certificate Manager (ACM)** in the same region as the Application Load Balancer.
2. Click **Request certificate** → **Request a public certificate**.
3. Add the domain names:
 - **chandra.quest**
 - **www.chandra.quest**
4. Select **DNS validation** and request the certificate.
5. ACM generates **CNAME Name** and **CNAME Value** records for domain validation.
6. Open **GoDaddy** → **Domain** → **DNS** → **DNS Records**.
7. Create the CNAME validation records provided by ACM.
8. Wait until the ACM certificate status changes from **Pending validation** → **Issued**.
9. Open **EC2** → **Load Balancers** → **LB-Application-ALB**.
10. Go to **Listeners and rules** → **Add listener**.
11. Configure:
 - Protocol: **HTTPS**
 - Port: **443**
 - Action: **Forward to ALB-Target-Group**
 - Certificate source: **From ACM**
 - Select the issued **chandra.quest** certificate.
12. Make sure the ALB Security Group allows **HTTPS TCP 443**.
13. Save the listener and test the application using HTTPS.

Screenshot 1: ACM certificate request showing `chandra.quest` and `www.chandra.quest`.

Screenshot 2: GoDaddy DNS showing the ACM CNAME validation records.

Screenshot 3: Your current screenshot showing **Pending validation**. Later, replace it or add another screenshot showing **Issued**.

Screenshot 4: ALB Listeners and rules showing `HTTPS:443` and the target group.

Then one final browser screenshot showing:

`https://chandra.quest`

with the application loading successfully.

Once it changes to **Issued**, immediately configure **HTTPS 443**.

The screenshot shows the AWS Certificate Manager (ACM) console. A blue banner at the top indicates a successful request for certificate ID `d64dce52-ed87-4a1e-ab77-2eb5c89fc1cc` with a status of 'Pending validation'. The left sidebar shows the navigation menu with 'ACME' expanded. The main content area displays the 'Certificate status' for the specified ID, showing it is 'Pending validation' and 'AWS managed'. Below this, a table lists the domains for which the certificate is being requested:

Domain	Status	Renewal status	Type	CNAME name	CNAME value
chandra.quest	Pending validation	-	CNAME	<code>_a431e10296b7c9222ab99536c6b8b7cb.chandra.quest.</code>	<code>75fb602m-validat</code>
www.chandra.quest	Pending validation	-	CNAME	<code>0733d112226f3370dd168a366d620bfa.www.chandra.qu</code>	<code>670d31c</code>

Buttons for 'Create records in Route 53' and 'Export to CSV' are visible above the table.

dcc.godaddy.com/control/portfolio/chandra.quest/settings?ventureId=95925e94-0089-431f-85b1-3365b0f0ba80&referrer=vh-quicklink&itc=mya_vh_buildwebsite_dashboard

Apps | YouTube | Google | 12 Beginner Python... | Learn Python | Learn Data Structur... | LeetCode - The Wor... | Movierulz | Watch B... | Slack

Domains

Dashboard

Domain

Website

Email

Store

Appointments

Marketing

Conversations

Customers

Users

Deals

Connect Your Domain in Minutes

Set up your domain with your website, email, or social media profile faster than ever with GoDaddy Airo.

Connect Domain

Need assistance? Visit our help center for more information.

Close All

Verify Domain Ownership

Success

Your DNS record has been updated successfully. Most DNS updates take effect within an hour, but could take up to 48 hours to update globally. You can track DNS propagation live by clicking next to the DNS record.

Success

Your DNS record has been updated successfully. Most DNS updates take effect within an hour, but could take up to 48 hours to update globally. You can track DNS propagation live by clicking next to the DNS record.

9 records

Type	Name	Data	TTL	Actions
☐	A	@	WebsiteBuilder Site	1 Hour
☐	NS	@	ns39.domaincontrol.com.	1 Hour
☐	NS	@	ns40.domaincontrol.com.	1 Hour
☐	CNAME	www	chandra.quest.	1 Hour

0733d112226f3370

aws | Search | [Alt+S] | United States (N. Virginia) | chandrashekar (986881030460) | chandrashekar

AWS Certificate Manager > Certificates > d64dce52-ed87-4a1e-ab77-2eb5c89fc1cc

AWS Certificate Manager (ACM)

List certificates

Request certificate

Import certificate

AWS Private CA

▼ ACM

ACME certificates [New](#)

Endpoints [New](#)

d64dce52-ed87-4a1e-ab77-2eb5c89fc1cc [Delete](#)

Certificate status

Identifier
d64dce52-ed87-4a1e-ab77-2eb5c89fc1cc

ARN
 arn:aws:acm:us-east-1:986881030460:certificate/d64dce52-ed87-4a1e-ab77-2eb5c89fc1cc

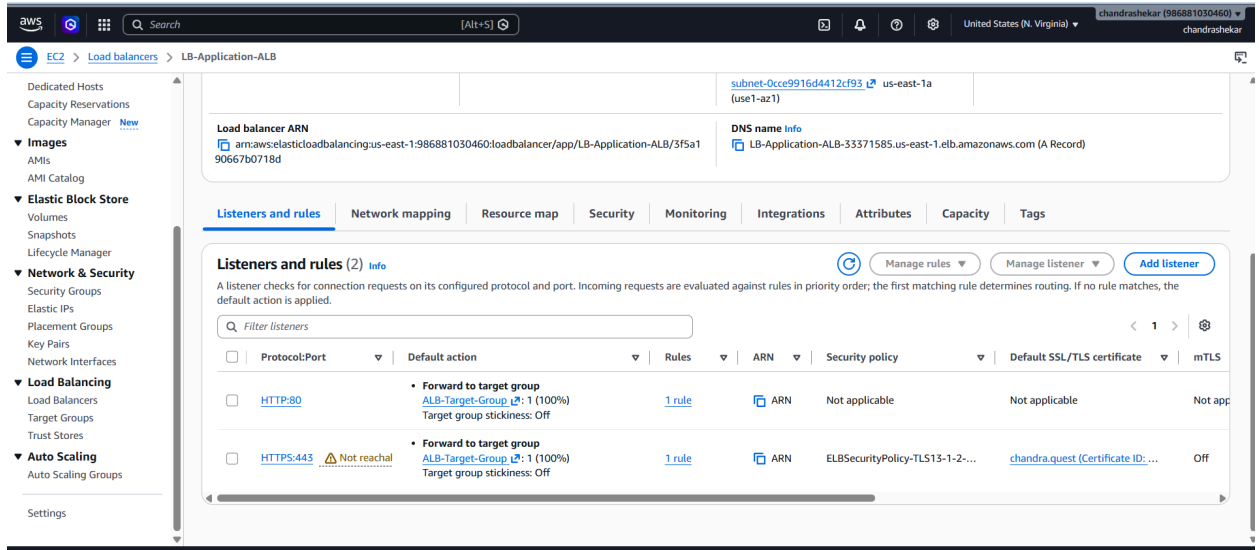
Type
Amazon Issued

Status
 Issued

Key source
AWS managed

Domains (2) [Create records in Route 53](#) [Export to CSV](#)

Domain	Status	Renewal status	Type	CNAME name	CNAME value
chandra.quest	Success	-	CNAME	_a431e10296b7c9222ab99536c6b8b7cb.chandra.quest.	_75fb602m-validation
www.chandra.quest	Success	-	CNAME	_0733d112226f3370dd168a366d620bfa.www.chandra.quest.	_670d31e-validation



5. Map Application load balancer to R53.

Task 5 – Map Application Load Balancer to Route 53

Objective: Map the custom domain **chandra.quest** to the Application Load Balancer using Amazon Route 53.

1. Open **AWS Console** → **Route 53** → **Hosted zones**.
2. Open the hosted zone **chandra.quest**.
3. Click **Create record**.
4. Configure the root-domain record:
 - Record name: **blank** (root domain)
 - Record type: **A**
 - Alias: **ON**
 - Route traffic to: **Alias to Application and Classic Load Balancer**
 - Region: **US East (N. Virginia)**
 - Load Balancer: **LB-Application-ALB**
 - Routing policy: **Simple routing**
5. Click **Create records**.
6. Create another record for **www**:
 - Record name: **www**
 - Record type: **A**
 - Alias: **ON**
 - Target: same **Application Load Balancer**
7. Test the domain in browser:
 - **https://chandra.quest**
 - **https://www.chandra.quest**
8. Verify that the request reaches the ALB and your Apache page appears.

You also completed the additional best-practice configuration:

HTTP:80 → 301 Redirect → HTTPS:443 → ALB Target Group → EC2

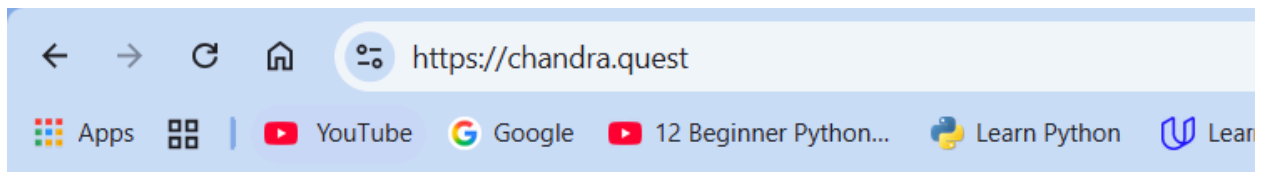
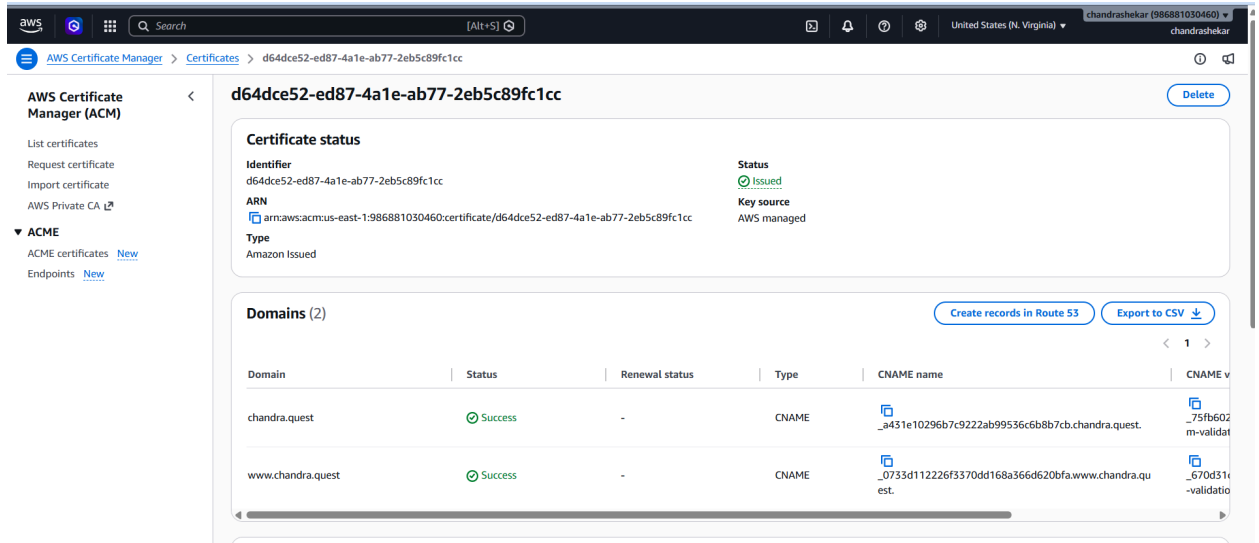
So Task 5 = COMPLETE ✓

The screenshot shows the AWS Route 53 console for the 'chandra.quest' hosted zone. A green notification banner at the top states: 'chandra.quest was successfully created. Now you can create records in the hosted zone to specify how you want Route 53 to route traffic for your domain.' The 'Hosted zone details' section shows the zone is public. The 'Records (2)' tab is active, displaying a table with two records:

Record	Type	Routing	Difference	Alias	Value/Route traffic to	TTL (s)	Health	Evaluation	Record ID
chandra.q...	NS	Simple	-	No	ns-1977.awsdns-55.co.uk, ns-1498.awsdns-59.org, ns-922.awsdns-51.net, ns-110.awsdns-13.com	172,800	-	-	-
chandra.q...	SOA	Simple	-	No	ns-1977.awsdns-55.co.uk, a...	900	-	-	-

The screenshot shows the AWS Route 53 console for the 'chandra.quest' hosted zone. A blue notification banner at the top states: 'Records for chandra.quest were successfully created. Route 53 propagates your changes to all of the Route 53 authoritative DNS servers within 60 seconds. Use "View status" button to check propagation status.' The 'Hosted zone details' section shows the zone is public. The 'Records (4)' tab is active, displaying a table with four records:

Record name	Type	Routing	Difference	Alias	Value/Route traffic to	TTL (s)	Health	Evaluation	Record...
chandra.quest	NS	Simple	-	No	ns-1977.awsdns-55.co.uk, ns-1498.awsdns-59.org, ns-922.awsdns-51.net, ns-110.awsdns-13.com	172,800	-	-	-
chandra.quest	SOA	Simple	-	No	ns-1977.awsdns-55.co.uk, a...	900	-	-	-
_a431e10296...	CNAME	Simple	-	No	_75fb6026732eeb08326963...	300	-	-	-
_0733d11222...	CNAME	Simple	-	No	_670d31c60469a8f3e88c6f8...	300	-	-	-



Load Balancer Lab

Response from: ip-10-0-1-228

Apache Web Server is running successfully.

6. Push the application load balancer logs to S3.

Task 6: Push Application Load Balancer Logs to S3

You can write these steps for your task/documentation:

1. **Create an S3 bucket**
 - Go to **S3** → **Create bucket**
 - Bucket name: **chandra-alb-access-logs-2026**
 - Keep the bucket in the same region as the ALB: **US East (N. Virginia) – us-east-1**
 - Click **Create bucket**
2. **Configure the S3 bucket policy**
 - Open the bucket → **Permissions** → **Bucket policy** → **Edit**

- Add a policy allowing the Elastic Load Balancing log-delivery service to write ALB logs into the bucket.
 - Save the policy.
- 3. **Open the Application Load Balancer**
 - Go to **EC2** → **Load Balancers**
 - Select **LB-Application-ALB**
- 4. **Open ALB attributes**
 - Select the **Attributes** tab.
 - Click **Edit**.
- 5. **Enable access logging**
 - Under **Logging**, enable **Access logs – legacy**
 - Set the S3 URI to:
- 6. `s3://chandra-alb-access-logs-2026`
 - Click **Save changes**.
- 7. **Verify the configuration**
 - AWS creates an object such as:

AWSLogs/
<AWS-account-id>/

8. ELBAccessLogTestFile

- You already received **ELBAccessLogTestFile**, so the ALB can successfully write to your S3 bucket. 
- 9. **Generate and verify actual logs**
 - Access/refresh <https://chandra.quest> several times.
 - Wait a few minutes.
 - Check the S3 bucket for actual ALB log files under a path similar to:

AWSLogs/
<AWS-account-id>/
elasticloadbalancing/
us-east-1/
2026/
09/
24/

10. [*.log.gz](#)

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Successfully modified load balancer attributes.

Introducing new Capacity Utilization metric for ALB

Monitor how much of your load balancer's available capacity was used to meet traffic demand. Visit the Capacity or Monitoring tabs to view utilization data.

LB-Application-ALB

Details

Load balancer type

Application

Status

Active

VPC

vpc-0978a6639ef05cc56

Scheme

Internet-facing

Hosted zone

Z35XDOTRQ7X7K

Availability Zones

subnet-01a6b737b03d4bb19 us-east-1b (use1-az2)

subnet-0cce9916d4412cf93 us-east-1a (use1-az1)

Load balancer IP address type

IPv4

Date created

September 24, 2026, 11:44 (UTC+05:30)

Load balancer ARN

arn:aws:elasticloadbalancing:us-east-1:986881030460:loadbalancer/app/LB-Application-ALB/3f5a190667b0718d

DNS name

LB-Application-ALB-33371585.us-east-1.elb.amazonaws.com (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Amazon S3

Buckets

Buckets

General purpose buckets

Directory buckets

General purpose buckets (2)

Find buckets by name

Name	AWS Region	Creation date
asg-alb-logs-986881030460	US East (N. Virginia) us-east-1	September 23, 2026, 15:26:38 (UTC+05:30)
chandra-alb-access-logs-2026	US East (N. Virginia) us-east-1	September 24, 2026, 15:32:27 (UTC+05:30)

Account snapshot

External access summary

Amazon S3

Buckets

chandra-alb-access-logs-2026

AWSLogs/

986881030460/

ELBAccessLogTestFile

ELBAccessLogTestFile

Properties

Permissions

Versions

Object overview

Owner

41f5a0213cf08661e91575dd0bedc50a405ccf61026a90f8bc14bfd86bc9818

AWS Region

US East (N. Virginia) us-east-1

Last modified

September 24, 2026, 15:40:47 (UTC+05:30)

Size

93.0 B

Type

Key

AWSLogs/986881030460/ELBAccessLogTestFile

S3 URI

s3://chandra-alb-access-logs-2026/AWSLogs/986881030460/ELBAccessLogTestFile

Amazon Resource Name (ARN)

arn:aws:s3:::chandra-alb-access-logs-2026/AWSLogs/986881030460/ELBAccessLogTestFile

Entity tag (Etag)

298dca9f110e4de9b1f2a3a71ee3fb75

Object URL

https://chandra-alb-access-logs-2026.s3.us-east-1.amazonaws.com/AWSLogs/986881030460/ELBAccessLogTestFile

Object management overview

The following bucket properties and object management configurations impact the behavior of this object.

Bucket properties

Management configurations

How to explain it in the morning session

“For Task 6, I configured Application Load Balancer access logging to Amazon S3. First, I created an S3 bucket and configured its bucket policy to allow the ELB log-delivery service to write objects. Then I opened my ALB attributes, enabled access logs, and configured the S3 bucket as the destination. After saving, AWS successfully created the ELB access-log test file in the bucket, confirming that the load balancer has permission to deliver logs to S3. Actual request logs are then stored in S3 for monitoring, troubleshooting, and analysis.”

Task 6: Push ALB logs to S3 — COMPLETE 